

ML, Signal, Image — Practical session 2

The sparse model

Clément Elvira

2023-2024

This second lab revolves around the “sparse model”. The first exercise (Task 1) is a continuation of the first lab session and should provide a better understanding of the sparse model. It is designed so that you can (and should) reuse the code you have already written. The second exercise (Task 2) is an application to image inpainting. It is (intentionally) less guided and will call on your modeling skills.

1 Task 1: getting started with the sparse model

A close look at the shape of capital letters reveals that they can (almost all) be drawn by combining elementary "symbols". The most striking example is probably provided by the letters "E", "F" and "L", which break down into: i) a vertical bar common to all three letters; ii) a horizontal bar on top for "E" and "F"; iii) etc. Note that the same vertical bar is also used to draw the letters "D", "K", "P" and "R".

With this observation in mind, it can be said that any capital letter can be represented as a combination of a few "elementary" symbols from a dictionary. The ultimate goal of this first exercise is thus to identify handwritten capital letters from a small number of linear measurements.

1.1 Introduction

1. Load the dictionary matrix available at “data_task_1/dic.csv”.
In python, you can use the function `numpy.loadtxt` (click on the function to access the documentation) from the Numpy library with the argument “`delimiter=‘, ’`”.

Each one of the 13 columns of the dictionary matrix represents a (vectorized) elementary symbol that can be used to represent a 28×28 pixels capital letters.

2. Plot the 13 elementary symbols. What kind of letters can you represent with them?
hint: there is (at least) 12 possible letters...

For simplicity's sake, we'll consider that a letter has been correctly identified if the support of the vector related to its sparse decomposition in the dictionary makes it possible to find all the elementary symbols necessary for its representation. To this end, we considered as recovery metric the F_1 score designed as

$$F_1(\mathcal{S}^*, \hat{\mathcal{S}}) = \frac{2P(\mathcal{S}^*, \hat{\mathcal{S}}) R(\mathcal{S}^*, \hat{\mathcal{S}})}{P(\mathcal{S}^*, \hat{\mathcal{S}}) + R(\mathcal{S}^*, \hat{\mathcal{S}})} \quad (1)$$

where $\mathcal{S}^*$ is the “true” support and $\hat{\mathcal{S}}$ is the identified one, and

$$P(\mathcal{S}^*, \hat{\mathcal{S}}) = \frac{\text{card}(\mathcal{S}^* \cap \hat{\mathcal{S}})}{\text{card}(\hat{\mathcal{S}})} \quad (2a)$$

$$R(\mathcal{S}^*, \hat{\mathcal{S}}) = \frac{\text{card}(\mathcal{S}^* \cap \hat{\mathcal{S}})}{\text{card}(\mathcal{S}^*)} \quad (2b)$$

are the recovery precision and recall, respectively. In particular, one has $F_1(\mathcal{S}^*, \hat{\mathcal{S}}) = 1$ when a perfect recovery of the support of x occurs and $F_1(\mathcal{S}^*, \hat{\mathcal{S}}) = 0$ when no elements of the support have been identified.

3. Implement a function that evaluates the F_1 score.

1.2 Recovery on a toy example

4. Generate a “R” using the dictionary of symbols.
you can use any values for the nonzero weights. I recommend setting them all to 1.
5. What kind of decoder could you use?
6. Describe the problem in terms of the four ingredients.
7. Implement a decoder that returns the support of the sparse representation.
 What do you conclude?
hint: you can use the Basis Pursuit solver provided with this tutorial, see the python file at data_task_1/bp.py.

We now consider the case where the 28×28 image is subsampled. To speed things up, let us consider directly the case where each one of the m "measurements" is obtained by taking the inner product of the image with a random Gaussian matrix.

8. Write the observation model.
9. From a theoretical point of view, how many measurements are (at least) needed to recover the support?
10. Set $m = 8$, generate of a (random Gaussian) sensing matrix and apply the proposed decoder. Plot the reconstructed image and evaluate the F_1 score. Compare the results with what you obtain using the subspace model.
11. Still considering the letter "R" generated above, plot the F1 score as a function of m . For each value of m , you for instance average the F_1 score over realizations of the random measurement matrix. Conclude.

1.3 Please Mr. Postman

In this last series of questions, the objective will be to apply our methodology to identify handwritten capital letters.¹

12. Load the dataset stored in `data_task_1/handwritten.csv`. In particular, each one of the 48 columns is a vectorized version of such an 28×28 image.
13. Plot a few of them.

Since perfect image reconstruction is most probably not possible, we can use the "denoising" version of the Basis Pursuit decoder recalled below.

$$\arg \min_{\mathbf{x} \in \mathbf{R}^n} \|\mathbf{x}\|_1 \quad \text{s.t.} \quad \|\mathbf{y} - \Phi \mathbf{x}\|_2^2 \leq \varepsilon. \quad (\text{BP-denoising})$$

where $\mathbf{y} \in \mathbf{R}^m$ is the measurement vector, $\Phi \in \mathbf{R}^{m \times n}$ is the dictionary matrix and ε is some parameter. Note however that solvers from the literature rather propose to solve the equivalent² problem referred to as "LASSO"

$$\arg \min_{\mathbf{x} \in \mathbf{R}^n} \frac{1}{2} \|\mathbf{y} - \Phi \mathbf{x}\|_2^2 + \lambda \|\mathbf{x}\|_1 \quad (3)$$

¹Extracted from <https://www.kaggle.com/datasets/sachinpatel21/az-handwritten-alphabets-in-csv-format/>

²that is for all $\varepsilon > 0$, there exists $\lambda_\varepsilon > 0$ such that the two problems shares the same set of minimizers, and conversely.

where $\lambda > 0$ is a positive scalar.

1. Implement a function that solve (3).
I recommend leveraging the `scikit-learn` library, see e.g. [this page of the documentation](#).
2. What kind of methodology have you seen in other courses to tune the value of λ ?
In this lab, I nevertheless suggest using default values of `scikit-learn`.
3. Try to identify a few digits. How performances could be improved?

2 Task 2: application in image inpainting

In this second exercise, we want to restore an image that has been damaged by a hard disk incident. Specifically, we'll assume that some pixel values are missing. The task will be to restore the missing pixel leveraging the sparse model.

Fortunately, it is well known in the image processing literature that natural images can be well approximated by their sparse decomposition in a well-chosen redundant dictionary. More precisely, we will use the fact that their “overlapping patch decomposition” admits a sparse decomposition. A patch is defined as a small piece of image. In this tutorial, we'll consider 8×8 patches. In this way, we can decompose an image by considering all the sub-images of size 8×8 , transform them and then recombine them to reconstruct a new image. The term “overlapping” refers to the fact a pixel can be present in several patches.

Resources:

- The deteriorated image `data_task_2/unknown.jpg`.
- The mask `data_task_2/mask.csv`. More precisely, it is a binary matrix with the same dimension as the image, and whose entries are 0 if the pixel is missing and 1 otherwise.
- A dictionary Φ `data_task_2/dictionary.csv`. It is a real matrix of size 64×256 .
- the module “feature_extraction” from the `scikit-learn` library. See [this page](#) for patches extraction and [this page](#) for patches recombination.

Remarks

- You can obviously leverage the LASSO-based decoder used in Task 1 (see equation (3)).
- It is known in the image processing literature that non-centered set of patches may induce strong biases in the reconstruction. A standard approach thus consists in first subtracting the “average patch” to all patches prior to any processing and adding it before recombination.